Instituto Superior de Formación Técnica Nº 151

Carrera: Analista en Sistemas

3 Año. Algoritmos y Estructuras de Datos III.

Trabajo Práctico Nº1	Unidad 1
Modalidad: Semi -Presencial	**Estratégica Didáctica:** Trabajo individual.
Metodología de Desarrollo: Det. docente	**Metodología de Corrección:** Via Classroom.
Carácter de Trabajo: Obligatorio – Con Nota	**Fecha Entrega:** A confirmar por el Docente.

Marco Teórico:

Responder el siguiente cuestionario en función de la bibliografía JpruebA Obligatoria.

1. Describir los Tipos de Patrones.

2. Que es un patrón de Software

3. Clasificar los Patrones de Software

4. clasificar los Patrones de Creación y realizar una Descripción

5. Dar un ejemplo de uso de cada uno.

Marco Practico:

1. **Patrón de Diseño Abstract Factory**

Abstract Factory es un patrón de diseño creacional, que te permite producir familias de objetos relaciondos con sus clases concretas. Para hacer más simple la explicación tomaremos el siguiente ejemplo.

Problema

Imagina que estás creando un simulador de una tienda de muebles. Tú código consiste en las clases que representan lo siguiente:

1. **Una familia de productos relacionados tal como: Chair + Sofa + CoffeTable.**
2. **La colleción de objetos puede tener varias combinaciones, por ejemplo: Chair + Sofa + CoffeTable que están disponibles en sus diferentes variaciones: Modern, Victorian, ArcDeco.**

Necesitas una forma de crear los objetos muebles individuales de tal forma que sea posible agruparse con los otros objetos de su misma familia, a razón es porque los clientes se enojan cuando no reciben una agrupación de muebles cuando lo requieren.

Considerando que tu no quieres cambiar el código existente cada que se añaden nuevos productos o familia de productos al programa y por su puesto esto no es para nada una buena práctica. Los vendedores de la mueblería actualizan los catálogos con mucha frecuencia y tu no debes hacer cambios al código cada vez que se esto sucede.

Implemetar la Solución

1. Se Solicita Diseñar una Aplicación en C++ que de respuesta a este problema.

Material Xtra.

Lo primero que el patrón Abstract Factory sugiere es que explicitamente las interfaces se declaren por cada producto distinto de la familia del producto. Después puedes hacer las variantes de los productos siguiendo estás interfaces. Por ejemplo, todas las variantes que pueden implementar la interface Chair

